WEEK 1 · TECHNICAL REPORT

LLM Foundations

Building a provider-agnostic API with FastAPI, Ollama and Groq

Project: **Tunisian Admin Assistant**, a future RAG chatbot on Tunisian labor law, CNSS and company creation

9	2	~10×
steps completed	LLM providers	faster tokens/s (cloud)

What this report covers

- Every step of the week, in order: goal, theory, what was built, real results.
- The core LLM concepts: tokens, context window, temperature, prompts, structured output, local vs API.
- The engineering choices: typed config, async calls, error handling, provider abstraction, Docker.
- The incidents we hit (hallucinations, a retired model, a leaked key) and what they taught.

Author Mohamed Amir Essmin

Date October 10, 2026
Repository github.com/miromiro007/tunisian-admin-assistant

Contents

Context and objectives 3

Step 0 · Environment and model selection 3

Step 1 · Professional project skeleton 4

- Dependencies . . . 4
- Configuration and secrets . . . 4
- Logging and the FastAPI entry point . . . 4

Step 2 · Tokens: measuring the cost of a language 5

Step 3 · Context window and temperature 5

- Context window . . . 5
- Temperature and top_p . . . 6

Step 4 · Prompts, schemas and `POST /chat` 6

Step 5 · Structured JSON output 7

Step 6 · Provider abstraction: Ollama + Groq 7

Step 7 · Comparing the two models 8

Step 8 · Docker Compose and GitHub 9

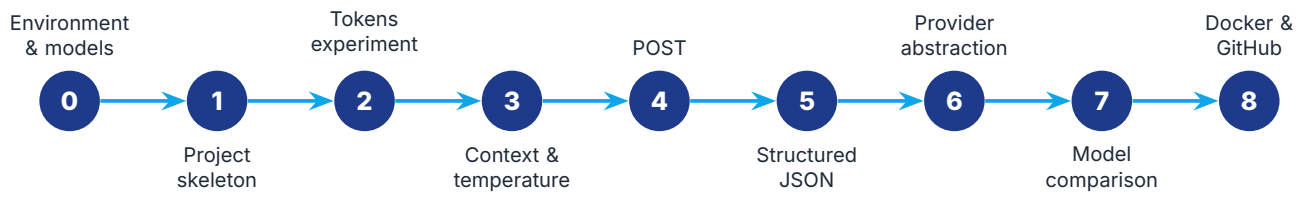
Glossary of technical terms 10

Interview preparation 10

Lessons, checklist and next steps 11

- What I learned . . . 11
- Checklist . . . 12
- Next: Week 2 . . . 12

The week at a glance



Context and objectives

The long-term goal is a **RAG chatbot** (Retrieval-Augmented Generation: the model answers from real documents and cites them) for Tunisian administrative questions. Week 1 deliberately stops before RAG and builds the foundations every LLM product needs.

Deliverable of the week

A FastAPI service that talks to a **local LLM** (Ollama, `qwen2.5:7b`) and a **hosted LLM** (Groq, `openai/gpt-oss-120b`), with a selectable provider, Pydantic-validated JSON output, a model comparison, Docker Compose, and a GitHub repository.

Teaching method. Each step followed the same pattern: goal, theory (simple definition, analogy, technical version), complete code, commands, expected result, common errors, quiz, interview question, and a Git commit.

STEP 0

Environment and model selection

Goal: size the local model to the hardware, then choose a free cloud API.

0

Component	Value	Impact
CPU	Intel i5-12500H	fine for the API, slow for CPU inference
RAM	16 GB	enough, Docker Desktop can take a large share
GPU	RTX 4060 Laptop, **8 GB VRAM** (75 W)	the key component for local LLMs
Software	Windows, Python 3.13.5, Docker 28.4, Compose 2.39	all ready

Theory To answer, a model keeps all its weights in fast memory (VRAM). Memory ≈ parameters × bytes per weight. At 16-bit, a 7B model needs ∼14 GB. **Quantization** stores each weight with fewer bits: **Q4** (≈ 4 bits) shrinks it to ∼4.7 GB. Add room for the **KV cache** (the model's working memory for the conversation) and ∼1 GB used by Windows.

Analogy VRAM is a kitchen counter. If all ingredients fit on it, the chef is fast. If not, the chef walks to the storage room (system RAM) and cooking becomes 5 to 10 times slower.

Size	Q4 memory	Fits in 7 GB?	Verdict
3B	∼2.5 GB	yes	very fast, weaker
7–8B	∼4.7–5 GB	**yes**	**sweet spot**
12–14B	∼8–9 GB	no	spills into RAM, 5–10× slower

Choices. Local: `qwen2.5:7b` (multilingual EN/FR/AR, good at JSON). Cloud: **Groq** (free tier, no credit card, OpenAI-compatible API). Keeping both is required by the program and is good engineering: privacy and offline use locally, quality and speed in the cloud.

Result `ollama ps` → `qwen2.5:7b 4.7 GB 100% GPU context 4096.` `nvidia-smi` → 4631 / 8188 MiB used. Ollama is a local HTTP server on `http://localhost:11434`.

STEP 1

Professional project skeleton

Goal: create a clean FastAPI structure with typed configuration, logging, secrets protection and a health route.

1

```
tunisian-admin-assistant/
|-- app/
|   |-- __init__.py, config.py, logging_config.py, main.py, schemas.py
|   `-- llm/  base.py, ollama_provider.py, groq_provider.py,
|             factory.py, prompts.py, errors.py
|-- scripts/  count_tokens.py, temperature_test.py, compare_models.py
|-- .env (ignored)   .env.example   .gitignore   requirements.txt
`-- Dockerfile   docker-compose.yml   .dockerignore   README.md
```

Dependencies

Package	Role in the project	.NET equivalent
fastapi	routes, validation, automatic Swagger docs	ASP.NET Core Web API
uvicorn	ASGI server that listens for HTTP requests	Kestrel
pydantic	validates requests, responses and LLM output	DTO + DataAnnotations
pydantic-settings	typed settings from `.env`	`IOptions<T>`
httpx	async HTTP client to call Ollama and Groq	`HttpClient`
tiktoken	counts tokens (approximation)	–

Configuration and secrets

Theory Configuration is separated from code (**12-factor** principle). `.env` holds real values and is ignored by Git; `.env.example` holds placeholders and is committed. `BaseSettings` reads each variable, converts and validates its type at startup (**fail fast**). `SecretStr` prints `**********`; the real key is only read with `.get_secret_value()` where it is sent. `@lru_cache` builds settings once (singleton). Priority: real env variable > `.env` > default in code.

```python
class Settings(BaseSettings):
    ollama_base_url: str = "http://localhost:11434"
    ollama_model: str = "qwen2.5:7b"
    groq_api_key: SecretStr = SecretStr("")   # hidden in logs
    groq_model: str = "openai/gpt-oss-120b"
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

@lru_cache
def get_settings() -> Settings:
    return Settings()
```

Watch out Fill `.gitignore` **before** creating `.env`. Check with `git check-ignore .env`. A typo such as `GROQ_API_KEYY` does not crash: `extra="ignore"` drops it and the key stays empty.

Logging and the FastAPI entry point

Logs go to **stdout** with the format `time | level | module | message`, so Docker can collect them. Levels: `DEBUG < INFO < WARNING < ERROR < CRITICAL`; production uses INFO, and secrets are never

logged. `main.py` creates the app, a `lifespan` hook (code before `yield` runs at startup, after it at shutdown) and `GET /health`, a liveness probe for Docker and load balancers.

Result `Starting Tunisian Admin Assistant (env=dev),` `GET /health 200 OK` → `{"status":"ok","environment":"dev"}`, Swagger UI at `/docs`.

.NET link `main.py` ≈ `Program.cs`, `@app.get` ≈ `[HttpGet]`, `lifespan` ≈ `IHostedService`, `.venv` ≈ a per-solution package set.

STEP 2

Tokens: measuring the cost of a language

2

Goal: see how the same sentence is split in English, French and Arabic.

Theory An LLM reads **tokens**, not words. A **tokenizer** (usually **BPE**, Byte Pair Encoding) builds a vocabulary by merging the most frequent byte pairs of its training data. Each token becomes an integer ID. Cost, speed and context window are all counted in tokens.

Analogy LEGO bricks: common words are one big brick, rare words are built from several small ones.

Language	Characters	Tokens	Chars/token
English	39	9	4.33
French	41	8	5.12
Arabic	30	**25**	**1.20**

How the text was cut: English keeps whole words (`' register'`, `' company'`); French splits `' Tunis' + 'ie'`; Arabic is cut almost letter by letter, and the last character even into two raw bytes (shown as `\ufffd`). Only very common words such as “in” stay whole.

Why it matters Arabic costs ∼2.8× more tokens than English for the same meaning, because the tokenizer’s training data is English-dominated. For a Tunisian assistant this means higher cost and a context window that fills faster, which will shape document chunking in RAG. `tiktoken` is OpenAI’s tokenizer, so for Qwen and Llama it is only an approximation: exact counts come from the API responses.

STEP 3

Context window and temperature

3

Goal: understand the model’s memory limit and its randomness dial, then test them.

Context window

The maximum number of tokens the model sees at once: prompt + history + answer. Beyond it, old content is **truncated** (forgotten) or the request fails. An LLM is **stateless**: it remembers nothing between calls, so a chatbot must resend the whole history every time. Ollama uses a 4096-token default window.

Temperature and top_p

At each step the model assigns a probability to every possible next token. **Temperature** reshapes them: low means the top choice dominates, high flattens them. **top_p** keeps only the most likely tokens whose probabilities sum to p. Change one, not both.

Task	Temperature
Extraction, classification, legal/admin facts	**0 – 0.2**
General Q&A	0.3 – 0.7
Creative writing, brainstorming	0.8 – 1.2

Result Prompt "Give me a funny name for a Tunisian cafe", 3 runs each, through `httpx`:
$T = 0$: *Carrot Cafe Caravan* three times, identical (in=40, out=43).
$T = 1.2$: *Carrot Café Caravan*, *Desert Dreams Café*, *Carpet Café Coffee* (out = 38, 26, 35), one even drifting to "Moroccan".
A 10-word prompt cost **40 input tokens**: Ollama wraps it in Qwen's chat template (hidden role markers).

Watch out In PowerShell, `curl` is an alias of `Invoke-WebRequest`. Use `cmd`, `Invoke-RestMethod`, or simply a Python script with `httpx`.

STEP 4
Prompts, schemas and `POST /chat`
Goal: send a question to the LLM through our own API, with validated input and output.

4

Theory Chat APIs take a list of messages with **roles**: **system** (developer rules: role, language, "never invent laws"; highest priority, unseen by users), **user** (the question), **assistant** (previous answers, resent as history). **Few-shot** examples improve format consistency.

Analogy A restaurant: the system message is the manager's briefing, the user message is the customer's order.

```python
class ChatRequest(BaseModel):
    question: str = Field(..., min_length=1, max_length=2000)
    provider: Literal["ollama", "groq"] = "ollama"
    temperature: float = Field(default=0.2, ge=0.0, le=2.0)
```

An empty question is rejected with **HTTP 422** before any token is spent; `max_length` protects against huge, costly prompts. The route calls the provider with `httpx.AsyncClient`: **async** lets the server keep serving other users while it waits. Failures are mapped to clean statuses: connection refused → **503**, timeout → **504**, provider error → **502**.



Result "How do I create a company in Tunisia?" → 200 OK, `prompt_tokens`=69, `completion_tokens`=253, `latency_ms`=18366 (first call includes loading the model into VRAM).

Watch out **Hallucination.** The answer sounded confident but was partly wrong (open a bank account "with your CNSS number", CNSS registration before the company exists). A 7B model has limited Tunisian knowledge: the main argument for RAG with cited sources.

STEP 5
Structured JSON output
Goal: turn a question into validated data: `theme`, `intent`, `keywords`.

5

Theory Free text cannot feed other code; JSON can. Three layers of reliability: (1) a clear prompt with **few-shot** examples at temperature 0; (2) **constrained decoding**, Ollama's `format` receives a JSON Schema generated by Pydantic; (3) **Pydantic validation** with `model_validate_json`. We never trust an external system (**defense in depth**).

```
class QuestionAnalysis(BaseModel):
    theme: Literal["labor_law", "cnss", "company_creation", "taxes", "other"]
    intent: str = Field(..., min_length=1, max_length=200)
    keywords: list[str] = Field(..., min_length=1, max_length=8)
```

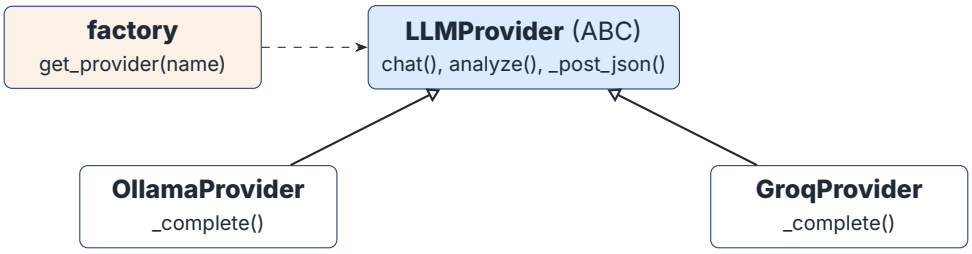
`Literal` works like an `enum`: a reply such as `"theme":"legal"` raises a `ValidationError`, which becomes a clean 502 and never reaches the rest of the system.

Result "Quels documents faut-il pour créer une SARL en Tunisie ?" → `theme: company_creation`, intent "Identify required documents for creating a SARL in Tunisia", keywords in English although the question was French: small models follow instructions only partially.

STEP 6
Provider abstraction: Ollama + Groq
Goal: support two interchangeable providers without duplicating code.

6

Theory An abstract class `LLMProvider` defines the common "plug shape". With the **template method** pattern, `base.py` implements `chat()` and `analyze()` once (timing, logging, error mapping, validation) and each provider implements only `_complete()`, the HTTP part that differs. A **factory** `get_provider(name)` returns the right object. Adding Gemini means one new class, one factory line, one `Literal` value: the **open/closed principle**.



	Ollama	Groq (OpenAI-compatible)
URL	`{base}/api/chat`	`.../openai/v1/chat/completions`
Auth	none	`Authorization: Bearer <key>`
Answer	`message.content`	`choices[0].message.content`
Tokens	`prompt_eval_count`, `eval_count`	`usage.prompt_tokens`, `completion_tokens`
JSON	`format` = full JSON Schema	`response_format: json_object` (valid JSON, not our schema)

Watch out **Retired model.** `llama-3.3-70b-versatile` returned **404 model_not_found**. Listing `/openai/v1/models` with our key showed the models available; we switched to `openai/gpt-oss-120b` by editing one line of `.env`. Model names are configuration, not code.

Result Groq `/chat`: 129 prompt / 831 completion tokens, **2870 ms**. Groq `/analyze`: keywords in French (`créer`, `Tunisie`), **1322 ms**. Richer answers, but still unverified legal details.

STEP 7

Comparing the two models

7

Goal: measure speed, tokens and quality on the same 8 questions.

Theory A fair **evaluation**: same prompts, same temperature (0), a **warm-up** request first (to exclude model loading into VRAM), and **tokens/second** rather than total latency (which depends on answer length). Questions mixed EN/FR/AR plus one off-topic control. Quality needs human judgment against official sources.

Model	Avg latency	Tokens/s	Prompt tok.	Completion tok.
`qwen2.5:7b` (local)	4203 ms	25.0	71	128
`gpt-oss-120b` (Groq)	1828 ms	**240.9**	130	490



Why it matters Groq generates ∼10× more tokens per second, but total latency is only ∼2.3× lower because its answers are ∼4× longer, partly due to hidden reasoning tokens that are billed. The two tokenizers count the same prompt differently (71 vs 130). Eight questions give an indication, not a scientific benchmark.

	Local (Ollama)	API (Groq)
Privacy	data stays on the machine	data sent to a third party
Cost	free, unlimited	free quota, rate limits (HTTP 429)
Offline	yes	no
Quality / speed	7B, GPU-dependent, ∼25 tok/s	∼120B, ∼240 tok/s

STEP 8

Docker Compose and GitHub

8

Goal: run API + Ollama with one command, then publish the code safely.

Theory **Image** = the package, **container** = a running instance. The **Dockerfile** builds the API (Python 3.12-slim, requirements first for layer caching, non-root user, `--host 0.0.0.0`). **Compose** runs `api` and `ollama` on one network where **service names are hostnames**: the API calls `http://ollama:11434`, because inside a container `localhost` means the container itself. A **volume** keeps the 4.7 GB model files. Secrets are injected at runtime with `env_file`, and `.dockerignore` keeps `.env` out of the image. A **healthcheck** calls `/health`; the GPU is passed with an NVIDIA device reservation.

Analogy A food court: each stall (container) has its own kitchen, they share one building with an internal phone line (the Compose network).

Watch out **Secret in Git history.** A real Groq key ended up in `.env.example`. GitHub **push protection** blocked the push (`GH013`). Deleting the file in a new commit did not help, because a push sends the whole history. Fix: revoke the key, restore placeholders, re-create the history, check with `git grep "gsk_"`, then push. Never click “allow the secret”.

Result Push succeeded: `main -> main`. Docker Compose files are written; running the stack (`docker compose up -d --build`) remains to be verified.

Glossary of technical terms

Term	Meaning
LLM	Large Language Model, predicts the next token from the previous ones
Token / tokenizer / BPE	unit of text read by the model / tool that splits text / Byte Pair Encoding algorithm
Context window	max tokens visible at once (prompt + history + answer)
Stateless	the model keeps nothing between requests
Temperature / top_p	randomness of sampling / nucleus sampling cut-off
System / user / assistant	message roles: developer rules / question / previous answers
Few-shot	examples inside the prompt to show the expected format
Hallucination	plausible but false content produced by the model
Structured output	answer forced into a fixed schema (JSON)
Constrained decoding	generation restricted to a JSON Schema
Quantization (Q4)	storing weights with fewer bits to save memory
VRAM / KV cache	GPU memory / working memory of the conversation
Latency / tokens per second	total response time / generation speed
RAG	Retrieval-Augmented Generation, answering from retrieved documents
ASGI / uvicorn	async Python web server interface / its server
DTO	Data Transfer Object, a request or response class
SecretStr	Pydantic type that hides secrets when printed
Fail fast	crash at startup on invalid configuration
ABC / template method / factory	abstract class / shared algorithm with one varying step / object creator
Open/closed principle	open to extension, closed to modification
Healthcheck / liveness probe	route used by platforms to know the app is alive
Volume	persistent storage outside a container
Push protection	GitHub feature that blocks pushes containing secrets

Interview preparation

Interview question

Q. *Why does Arabic cost more tokens than English?*

A. The tokenizer vocabulary is learned mostly from English-heavy data, so Arabic words are split into many small pieces, sometimes raw bytes. In our test the same sentence cost 25 tokens in Arabic versus 9 in English, which affects cost and context usage.

Interview question

Q. *Why set temperature to 0 for some tasks?*

A. Temperature controls sampling randomness. For extraction, classification or factual answers I want the most probable token every time, so outputs are stable, testable and less likely to drift.

Interview question

Q. *How do you get reliable structured output from an LLM?*

A.A few-shot prompt at low temperature, constrained decoding with a JSON Schema when supported, then Pydantic validation, because JSON mode guarantees valid JSON but not my schema. On failure I log the raw output and return a clear error.

Interview question

Q. *How would you support several LLM providers?*

A.An abstract provider interface, one class per vendor, shared logic in the base class, and a factory. Routes depend on the abstraction, so adding a provider is one class. It avoids vendor lock-in and makes comparison and fallback easy.

Interview question

Q. *How do you handle secrets, locally and in Docker?*

A.Config separated from code: `.env` ignored by Git, `.env.example` with placeholders, `SecretStr` so keys never reach logs. In Docker, never bake secrets into the image: `.dockerignore` and runtime injection. If a key leaks, revoke it first, then clean the history.

Interview question

Q. *How do you choose between a local model and a hosted API?*

A.Evaluate both on the same task set with fixed temperature: latency, tokens/s, cost, quality. Then weigh privacy, offline needs, rate limits and hardware. A hybrid that routes by sensitivity or difficulty is often best.

Lessons, checklist and next steps

What I learned

- LLMs read tokens; Arabic is ∼2.8× more expensive than English in our test.
- The context window is shared by prompt and answer; history must be resent because LLMs are stateless.
- Temperature 0 gives repeatable output; 1.2 gives variety and drift.
- System prompts set the rules; few-shot examples fix the format.
- Never trust LLM output: schema + Pydantic validation.
- Local (private, free, ∼25 tok/s) vs cloud (∼240 tok/s, richer, rate-limited).
- Model names belong in configuration: providers retire models.
- Both models hallucinate legal details: RAG with sources is necessary.
- A secret in Git history stays there: revoke first, then rewrite.

Checklist

Item	Status
`.env` ignored, `.env.example` with placeholders	done
Typed config with `SecretStr`, logging to stdout	done
`GET /health`, `POST /chat`, `POST /analyze`	done
Ollama and Groq providers, factory	done
Comparison on 8 questions	done
Repository pushed to GitHub	done
Docker Compose stack verified running	to verify
Exposed Groq key revoked	to confirm
Human quality scores (1–5) per answer	to add

Next: Week 2

Introduce RAG: **embeddings** (text turned into vectors), a **vector database**, document **chunking** (with Arabic token costs in mind) and **source citations**, so the assistant answers from official Tunisian texts instead of its memory.